

LangChain Fundamentals — Interview Questions

Reproduced exactly as originally written from LangChain/LangChain Fundamentals - Video Notes.md — nothing reworded.

Interview Prep — Mock Interview (LangChain Fundamentals, ~4 Years' AI Engineering Experience)

This set covers the fundamentals layer specifically — the five core components, package architecture, prompt templates, model abstractions, how memory mechanically works, and the "should you even use this framework" judgment call. It deliberately does not repeat the chain-vs-agent, LCEL-migration, tool-calling, structured-output, or LangGraph material already covered in Interview Prep/LangChain and LangGraph - Interview Questions.md. Curated from current (2026) LangChain interview question banks and production post-mortems, calibrated to roughly four years of AI engineering experience. Same two-layer format used throughout these notes: a plain-language answer with an everyday example (and a diagram wherever a picture helps), then a crisp, technically precise version worth saying out loud in the room. Try answering out loud before reading the model answer.

 **Interview Q1:** "LangChain's original pitch was that a plain LLM API call can't chain steps, can't remember anything, and can't reach the outside world. In 2026, do you actually still need a framework for those three things?"

✓ **Strong answer:** "Honestly — for those three things alone, no, and that's the interesting part of the question. Chaining two prompts is a variable and a second function call. 'Memory' is a list you append to. Calling an API is `requests.get`. None of that needs a framework, and in 2026 the provider SDKs have converged enough that the abstraction hides much less than it used to.

Where LangChain still earns its place is when you need *many* of those things at once, swappable, with the surrounding machinery already built: standardized interfaces across a dozen model providers, a couple hundred pre-built document loaders and vector store integrations, streaming and batching and async inherited for free, and tracing that already understands the shape of your pipeline.

Everyday example: it's like asking whether you need a web framework to serve HTTP. You can absolutely open a socket and write the response bytes yourself — for one endpoint, that's genuinely simpler. The framework pays off at endpoint fifty, when routing, middleware, sessions, and error handling would otherwise all be hand-rolled and subtly inconsistent.

So the honest answer isn't 'yes you need it' or 'no it's bloat' — it's that the three problems from the intro pitch are the *motivation*, not the *justification*. The justification is integration breadth and the production plumbing around it."

 **Standard Interview Answer:** "The three original motivations — sequential composition, conversational state, and external tool/data access — are each individually trivial to implement directly against a provider SDK, and since vendor APIs have largely converged, the abstraction layer hides fewer meaningful differences than it did in 2023. LangChain's remaining value proposition is breadth and standardization rather than capability: a uniform `Runnable` interface across providers, a large integration surface (document loaders, vector stores, retrievers, tool wrappers), and inherited cross-cutting behavior — streaming, batching, async, callbacks, and tracing — applied uniformly to any composed pipeline. The framework decision should therefore be driven by integration surface area and observability needs, not by whether chaining or memory is technically possible without it."

 **Interview Q1 (follow-up):** "Then be concrete — when would you deliberately *not* use LangChain?"

✓ **Strong answer:** "Four situations, pretty clear-cut:

One — a single LLM call with a prompt template. A full chain adds imports, a mental model, and debugging friction to something that's genuinely three lines against the raw SDK. If there's one call and no composition, the framework is pure overhead.

Two — latency-sensitive, high-throughput paths. The framework layer isn't free; benchmarks put it in the tens of milliseconds of extra overhead per call. That's noise on a 2-second RAG pipeline and very much not noise on a service making thousands of calls a minute where you're defending a tight p99.

Three — when debuggability matters more than convenience. A LangChain stack trace in production routinely spans dozens of frames of internal framework code. Finding the line that actually broke is a genuinely different skill than normal Python debugging, and that cost lands on whoever is on call at 3am, not on whoever wrote it.

Four — non-Python/JS stacks. The ports to other languages lag the main libraries badly enough that you'd be building on a second-class citizen.

The rule I'd actually state: use it when you're gaining integrations and observability you'd otherwise hand-build. Skip it when you're only gaining syntax."

 **Standard Interview Answer:** "Framework adoption is a cost-benefit decision, not a default. It's contraindicated for: single-call workloads where composition provides no benefit; latency-critical high-QPS paths, where measured per-call framework overhead (roughly 30-80ms on standard agent paths) is material against a tight latency SLO; systems where debuggability is a primary operational requirement, since framework-internal stack depth substantially increases mean-time-to-diagnosis; and non-Python/JavaScript runtimes where integration parity lags. The heuristic is whether the framework is supplying integrations and cross-cutting infrastructure you'd otherwise implement — if it's only supplying syntax over a converged vendor API, the raw SDK is the better engineering choice."

 **Interview Q2:** "Why is LangChain split into `langchain-core`, `langchain-community`, and separate provider packages instead of being one library?"

✓ **Strong answer:** "Because abstractions and integrations have completely different stability and maintenance profiles, and bundling them meant one couldn't move without dragging the other.

Everyday example: think of an electrical system. The *socket standard* — the shape of the plug, the voltage — changes almost never, and everything depends on it. The *appliances* that plug into it change constantly: new ones ship weekly, old ones break, some are maintained by their manufacturer and some by hobbyists. You absolutely do not want your socket standard to get a breaking change because someone's toaster driver was updated.

langchain-core	→ the socket standard: base types (messages, documents, tools, retrievers) + the Runnable protocol. Zero third-party dependencies. Changes very rarely.
langchain-openai langchain-anthropic	→ first-party appliance: one provider, tightly maintained, versioned independently, tested against that provider's API.
langchain-community	→ the everything-drawer: hundreds of integrations, largely community-maintained, moves fast, varying quality.
langchain	→ the assembly layer: chains, agents, retrieval logic built on core's abstractions.
langchain-classic	→ the retirement home: legacy interfaces kept importable for backward compatibility.

The practical consequence people actually hit: installing just `langchain` and then being confused when a document loader import fails — because loaders live in `langchain-community`, which you didn't install."

🔗 **Standard Interview Answer:** "The split isolates stable abstractions from volatile integrations. `langchain-core` holds the base interfaces — chat models, messages, documents, tools, retrievers, vector stores, callbacks — plus the `Runnable` invocation protocol, and deliberately carries no third-party provider dependencies, so it can maintain a strong stability guarantee. Provider packages (`langchain-openai` , `langchain-anthropic`) are first-party, independently versioned, and tested against a single vendor API. `langchain-community` carries the long tail of community-maintained integrations with a faster, looser release cadence. `langchain-classic` holds deprecated interfaces for migration compatibility. This lets integration churn proceed without forcing breaking changes on the core abstraction layer, and lets a consumer depend on only the surface area they actually use rather than transitively pulling every integration's dependency tree."

🗣️ **Interview Q3:** "LangChain exposes both an `LLM` interface and a `ChatModel` interface. What's the actual difference, and which should new code use?"

✅ **Strong answer:** "The difference is the shape of the input and output. An `LLM` is text-in, text-out — you hand it a string, you get a string. A `ChatModel` is messages-in, message-out — you hand it a *list* of role-tagged messages (system, human, AI) and get a single AI message back.

Everyday example: an `LLM` is like a vending machine — one slot, put text in, text comes out, no notion of who's asking or what was said before. A `ChatModel` is like a group chat thread — every message is tagged with who said it, and the model can treat 'the system said this is the policy' very differently from 'the user said this is the policy.'

```
LLM:      "Summarize this: ..."      → "Here's the summary..."
          (str in, str out)

ChatModel: [ SystemMessage("You are a support agent, never quote prices"),
             HumanMessage("How much is the pro plan?"),
             AIMessage("I can't quote prices, but..."),
             HumanMessage("Why not?") ]   → AIMessage("Because...")
          (list[Message] in, Message out)
```

New code should use `ChatModel`, essentially always. Nearly every current model is chat-native, and the role structure is what makes system instructions, tool calls, and multi-turn context work properly. Passing a plain string into a chat model still works — it gets wrapped as a single human message — but you lose the ability to separate

instructions from user input, which matters both for output quality and for prompt-injection resistance."

 **Standard Interview Answer:** " `LLM` is the legacy text-completion interface: `str` in, `str` out. `ChatModel` is the message-based interface: a sequence of role-typed messages (`SystemMessage` , `HumanMessage` , `AIMessage` , `ToolMessage`) in, a single `AIMessage` out. Chat models are the current standard because contemporary provider APIs are chat-native, and the role structure is load-bearing — it's what carries system-level instructions distinctly from user content, and it's the transport for tool calls and tool results. Use `ChatModel` for new development; the string-input convenience path coerces to a single human message, which silently discards the instruction/content separation that both output steerability and prompt-injection mitigation depend on."

 **Interview Q4:** "Why use LangChain's prompt templates at all instead of just a Python f-string?"

 **Strong answer:** "For one prompt in one place, an f-string genuinely is fine — I wouldn't pretend otherwise. Templates start paying off when the prompt becomes a *thing the system owns* rather than a line of code.

Three concrete reasons:

They declare their inputs. A template knows it needs `{customer_name}` and `{issue_type}`, so a missing variable fails immediately with a clear error at format time — instead of an f-string silently interpolating `None` into your prompt and the model confidently answering about a customer named 'None.'

They're composable objects, not strings. A template is a `Runnable`, so it pipes directly into a model. It can be partially filled now and completed later, and it can be swapped or A/B-tested without touching the code around it.

They separate prompt from logic. The prompt becomes reviewable, versionable, and editable by someone who isn't going to touch the pipeline code.

Everyday example: it's the difference between writing a letter from scratch every time versus having a form letter with blanks. For one letter, writing it out is faster. For ten thousand letters where legal needs to review the wording and marketing keeps tweaking the greeting, the form wins decisively."

 **Standard Interview Answer:** "Prompt templates provide declared input variables with validation at format time, producing an explicit `KeyError`-style failure on a missing variable rather than silent interpolation of an unintended value. They are `Runnable` objects, so they compose directly into LCEL pipelines and support partial binding, few-shot example injection, and message-role structuring that a raw f-string can't express. Operationally, they decouple prompt content from application logic, which is what makes prompts independently versionable, reviewable, and swappable for A/B evaluation. For a single static prompt the abstraction is unnecessary; the value scales with prompt count, prompt volatility, and the number of people who need to modify prompts without touching pipeline code."

 **Interview Q4 (follow-up):** "What's the difference between `PromptTemplate` and `ChatPromptTemplate`, and what are partial variables and few-shot templates for?"

 **Strong answer:** " `PromptTemplate` produces a plain string — it's the counterpart to the old text-completion `LLM` interface. `ChatPromptTemplate` produces a *list of role-tagged messages*, which is what a `ChatModel` actually wants. Same reasoning as Q3: with a chat model, use the chat template, because that's what preserves the system/human/AI separation.

```
ChatPromptTemplate.from_messages([
    ("system", "You are a support agent for {product}. Never quote prices."),
    ("human", "{user_question}"),
])
```

Partial variables solve a real timing problem: some values are known early, some only at request time.

`{product}` is known at startup; `{user_question}` arrives per request. Partialing pre-fills the known ones and hands back a new template awaiting only the rest — so you're not re-passing `product` on every single call, and you can't forget to.

A partial can also be a *function*, which is the genuinely useful case — `{current_date}` bound to a callable gets evaluated fresh at each format, rather than being frozen at the moment the template was built. That's a classic subtle bug: a date baked in at import time that's still showing yesterday's date a day later.

`FewShotPromptTemplate` handles example-driven prompting: instead of pasting examples into a giant string by hand, you give it a list of examples and a formatter, and it assembles them consistently. The payoff is that examples become *data* — you can swap the example set, or use an example selector to pick the most semantically relevant examples for each specific input rather than always sending the same fixed ones."

🔗 **Standard Interview Answer:** "`PromptTemplate` renders to a string for text-completion interfaces; `ChatPromptTemplate` renders to a structured message list preserving role separation, and is the correct choice for chat models. **Partial variables** support staged binding — pre-filling values known at construction time and returning a new template requiring only the remaining runtime variables — which reduces per-call argument plumbing and prevents omission errors. Partials accept callables, which are evaluated at each format invocation rather than bound once, correctly handling time-dependent values like current date that would otherwise be frozen at template construction. `FewShotPromptTemplate` externalizes in-context examples into structured data with a consistent formatter, enabling example-set versioning and dynamic example selection — typically semantic similarity against the incoming input — instead of a statically embedded example block."

🗣️ **Interview Q5:** "An LLM is stateless — every API call is independent. So mechanically, how does LangChain 'memory' actually make a chatbot remember?"

✅ **Strong answer:** "It doesn't make the model remember — that's the key insight. The model is stateless and stays stateless. Memory is just *text that gets re-injected into the prompt* on every single call.

Everyday example: it's like talking to someone with no short-term memory who reads a notebook before answering each question. They're not remembering — you're handing them the transcript every time, and they read it fresh. The illusion of continuity lives entirely in what you put in front of them.

Turn 3, what ACTUALLY gets sent to the API:

[SystemMessage]	"You are a support agent..."	
[HumanMessage]	"I contacted you last week about a refund"	← replayed
[AIMessage]	"I see that ticket, it's still open"	← replayed
[HumanMessage]	"Any update?"	← the new turn

The model sees all of it as one fresh, stateless request.

Three consequences fall directly out of that, and this is what the question is really probing:

Cost and latency grow with conversation length, because you're literally re-sending the whole history every turn — you pay for turn 1's tokens again on turn 50.

It eventually hits the context window, which is why summarizing and windowed strategies exist at all.

Memory is prompt content, so it's an injection surface. Anything a user said earlier is replayed into the prompt on every subsequent turn — if a user plants instructions in turn 2, those instructions are still sitting in the context at turn 20."

 **Standard Interview Answer:** "Memory is a prompt-construction concern, not model state. The model remains stateless across invocations; memory components persist prior turns externally and re-inject them into the message list on each call, so apparent conversational continuity is entirely a function of replayed context. This yields three direct consequences: token cost and latency scale with accumulated history since prior turns are re-transmitted every request; the context window imposes a hard ceiling, motivating windowed and summarization strategies; and replayed user content constitutes a persistent prompt-injection surface, since adversarial instructions introduced in an early turn remain in-context for every subsequent turn until explicitly evicted."

 **Interview Q5 (follow-up):** "Is that still how you'd build it in 2026?"

 **Strong answer:** "The mechanism is identical — it's still 'replay history into the prompt,' and that part is physics, not a design choice. What's changed is *where the history lives and who owns it*.

The legacy memory classes attached to a chain object and mostly kept history in process memory, which meant it evaporated on restart and didn't survive across multiple workers. The current direction is LangGraph's persistence layer: state lives in a checkpointing backed by something real — SQLite, Postgres, Redis — keyed by a thread ID.

Why that's a genuine improvement, not just churn: it survives process restarts, it works when requests are load-balanced across instances, it isolates concurrent conversations properly, and the same mechanism gives you pause-and-resume and human-in-the-loop for free, because 'resume this conversation' and 'resume this paused workflow' turn out to be the same primitive.

The practical advice I'd give: reach for durable, keyed persistence from day one rather than in-process memory, because 'this conversation is only three turns' stops being true the moment real users start talking."

 **Standard Interview Answer:** "The mechanism is unchanged — history replay into the prompt — but the persistence substrate has moved. Legacy chain-attached memory classes typically held state in-process, which fails on restart and under multi-replica deployment. Current practice uses LangGraph's checkpointing abstraction, persisting thread-keyed state to a durable backend (SQLite, Postgres, Redis). This provides restart durability, correct isolation across concurrent threads, horizontal-scaling compatibility, and — since interrupt/resume and conversational continuity share the same persistence primitive — human-in-the-loop and pause/resume semantics without additional machinery. Durable keyed persistence should be the default from initial implementation rather than a later migration."

 **Interview Q6:** "Take the support-chatbot example — it remembers a user's past tickets, adapts its tone by issue type, and looks up live order data. Map each of the five core components onto that, and tell me which one breaks first in production."

 **Strong answer:** "The mapping:

Memory	→ recalls "I contacted you last week about a refund"
Prompts	→ a template that swaps instructions by issue type (technical troubleshooting vs. order inquiry)
Models	→ the LLM generating the actual reply
Agents	→ decides per query whether to call an API or just answer
Chains	→ the pipeline wiring all of the above into one request path
Tools	→ the order-lookup / tracking API the agent can call

What breaks first: the tools, by a wide margin. Everything else is your own code in your own process. The tool call is a network boundary to a system you don't control, and it's where reality intrudes — the order API times out, returns a 500, returns a shape you didn't expect, or returns an empty result that isn't an error but isn't an answer either.

The specific failure that catches people isn't the tool failing — it's the *agent's reaction* to it failing. A naive agent sees an error, decides to try again, gets another error, tries again. Now you have a retry loop burning tokens against a dead endpoint, and the user is watching a spinner.

Second place: unbounded memory, which doesn't fail loudly at all — it just gets slower and more expensive every turn until a long conversation silently blows the context window.

What I'd actually build in: timeouts and a hard retry cap on every tool, an explicit 'I couldn't reach the order system, here's what I can tell you' fallback path, and a memory strategy with a bound on it from the first version."

 **Standard Interview Answer:** "Component mapping: memory persists prior ticket context; prompt templates branch instruction content on issue classification; the chat model generates the response; tools expose the order/tracking API; the agent performs per-query routing between direct response and tool invocation; the chain composes these into a single request path. The dominant first failure is the tool boundary, since it's the only component crossing a process and network boundary into an uncontrolled dependency — timeouts, upstream 5xx, schema drift, and semantically-empty successful responses. The compounding risk is the agent's error-handling policy rather than the tool failure itself: without a bounded retry policy, tool failure produces an unbounded reason-retry loop with linear token cost and no user-visible progress. Secondary failure is unbounded memory growth, which degrades cost and latency monotonically before failing at the context limit. Mitigations: per-tool timeouts and retry caps, an explicit degraded-response path on tool failure, and a bounded memory strategy from initial implementation."

 **Interview Q7:** "A chain is returning a bad answer and you can't tell which step caused it. How do you actually see inside it?"

 **Strong answer:** "In rough order of how much I'd reach for each:

LangSmith tracing, which is the real answer for anything non-trivial. You get a tree of the whole execution — every step's exact inputs and outputs, the fully-rendered prompt that actually reached the model, token counts, and per-step latency. The single most common revelation is that the prompt the model received wasn't the prompt you thought you wrote, because a variable was empty or the retrieved context was junk.

Callbacks, for programmatic access — handlers that fire on `on_llm_start`, `on_chain_end`, `on_tool_error` and so on. That's how you wire chain internals into your own logging, metrics, or cost tracking rather than eyeballing a UI.

Running sub-chains in isolation, which LCEL makes easy since every piece is independently invocable. Invoke just the prompt to see the rendered text. Invoke just the retriever to see what came back. Bisect until the bad step is obvious.

One trap worth naming: `verbose=True` on its own doesn't give you real visibility — it only bumps LangChain's internal logging chatter. People set it, see some output scroll past, and assume they have observability. They don't — it's not structured, it's not complete, and it's not something you can build alerting on.

Everyday example: `verbose=True` is a colleague muttering while they work. Tracing is a recording of the meeting with a transcript and timestamps. Only one of those helps you answer 'what exactly happened at 3:42.'"

🔗 **Standard Interview Answer:** "Primary tooling is LangSmith tracing, which materializes the full execution tree — per-step inputs and outputs, the fully-rendered prompt as transmitted, token accounting, and per-step latency — converting root-cause analysis from log interpretation into structured trace inspection. The most frequent finding is divergence between the intended prompt and the rendered prompt, caused by unpopulated template variables or low-quality retrieved context. For programmatic access, the callback system exposes lifecycle hooks (`on_llm_start`, `on_chain_end`, `on_tool_error`) for integration with existing logging, metrics, and cost-attribution infrastructure. Because LCEL components are independently invocable, sub-chain isolation is an effective bisection technique. Note that `verbose=True` is not an observability mechanism — it controls internal logging verbosity only, producing unstructured, incomplete output unsuitable for alerting or systematic diagnosis."

🎙️ **Interview Q8:** "How does streaming work in a LangChain pipeline, and what's the common gotcha?"

✅ **Strong answer:** "Any LCEL chain exposes `.stream()` because streaming is part of the shared `Runnable` interface — you don't build it per chain, you inherit it. Call it and you get tokens as they're produced instead of waiting for the full response.

The gotcha is that one blocking component silently kills streaming for the entire chain. Streaming only works end to end if *every* step can operate incrementally. The moment a step needs the complete input before it can produce any output, the stream stalls there and everything downstream waits.

Everyday example: it's a production line where one station insists on receiving the whole batch before starting. Every station after it sits idle regardless of how fast the ones before it were. Your users get the same wait they'd have had without streaming, plus you've now got streaming code that looks like it works.

The classic offender: an output parser that needs to parse complete, valid JSON. It genuinely cannot emit anything until the closing brace arrives, so a `prompt | model | json_parser` chain streams beautifully at the model and then buffers completely at the parser.

The other one that bites: the historical `streaming=True` constructor flag was removed years ago, so code copied from an old blog post sets a parameter that does nothing and appears to fail silently. `.stream()` is the mechanism now."

🔗 **Standard Interview Answer:** "Streaming is inherited from the `Runnable` interface — any composed LCEL chain exposes `.stream()` and `.astream()` without per-chain implementation. The constraint is that end-to-end streaming requires every component in the pipeline to support incremental processing; a single component requiring complete input before producing output serializes the entire downstream pipeline at that point, negating perceived latency benefit. The canonical case is a strict structured-output parser, which cannot emit until it receives syntactically complete input — so `prompt | model | json_parser` streams at the model boundary and then buffers at the parser. For partial structured output, streaming-aware parsers that emit progressively-completed partial objects are required. Additionally, the legacy `streaming=True` constructor parameter was removed in LangChain 0.1.0; it's a no-op in current versions, and `.stream()` is the correct interface."

 **Interview Q9:** "Agents are pitched as the flexible component — they decide what to do dynamically. When is that flexibility actually a liability?"

✅ **Strong answer:** "Whenever you already know the right sequence of steps. If the workflow is 'validate input, look up the order, generate a response,' handing that to an agent means paying an LLM to rediscover a decision you already made — every single request, non-deterministically, with some failure rate.

Everyday example: it's hiring a consultant to decide which form to fill out, when there's only ever one form. You pay for the deliberation, you wait for it, and occasionally they pick the wrong form anyway.

The three costs, concretely:

Non-determinism — the same input can take different paths on different days, which makes testing genuinely hard and makes incidents hard to reproduce.

Unbounded cost and latency — an agent loop has no natural upper bound on iterations. A confused agent can call the same tool repeatedly, and without a hard cap you find out via the bill.

Blast radius — an agent that *can* call a destructive tool eventually *will*, on an input nobody anticipated. A fixed chain can only ever do what it was wired to do.

My rule: fixed chain when the steps are known; agent only when the input genuinely determines the path and you can't enumerate the branches ahead of time. And when it is an agent: a hard iteration cap, per-tool timeouts, restricted tool scope, and human approval before anything irreversible."

 **Standard Interview Answer:** "Agent autonomy is a liability whenever the control flow is knowable at design time. Delegating a deterministic sequence to an LLM router incurs per-request inference cost and latency to re-derive a fixed decision, with nonzero routing error. The specific costs are: non-determinism, which degrades testability and incident reproducibility; unbounded iteration, since the reason-act loop has no intrinsic termination guarantee and can produce runaway token cost without an explicit cap; and expanded blast radius, since any tool in the agent's scope is reachable on unanticipated inputs, whereas a static chain's action space is fixed at construction. Selection criterion: use a deterministic chain when the step sequence is enumerable, and reserve agents for genuinely input-dependent control flow. Required controls when agents are warranted: hard iteration limits, per-tool timeouts, least-privilege tool scoping, and human-in-the-loop gating before irreversible side effects."

What interviewers are really scoring for, across all of the above: - Whether you can justify *using* a framework on engineering grounds, rather than treating it as the default — and whether you can name concrete conditions where you'd skip it - Whether you understand that memory is prompt injection of history, not model state — this single insight explains cost growth, context limits, and a security surface all at once - Whether you reach for the right term naturally (`Runnable` , `ChatPromptTemplate` , partial variables, checkpointing, callbacks) instead of describing around it - Whether "how would you debug this" produces a *method* (trace it, isolate sub-chains, inspect the rendered prompt) rather than "add print statements" - Whether you name the operational controls unprompted — iteration caps, timeouts, bounded memory, least-privilege tools — since that's what separates someone who's shipped an agent from someone who's built a demo

Sources consulted while calibrating this section: - [Top 50 LangChain Developer Interview Questions and Answers 2026 — Index.dev](#) - [Top LangChain Interview Questions and Answers for 2026 — DataCamp](#) - [Top 30 LangChain Interview Questions and Answers \(2026\) — Hrist](#) - [LangChain in 2026: When to Use It, When to Skip It — Osher Digital](#) - [The LangChain Exit: Why Production Teams Are Quietly Rewriting to Raw SDKs in 2026 — Ravoid](#) - [When You Should NOT Use LangChain — The Neural Base](#) - [OpenAI API vs LangChain: Real Overhead Benchmark \(2026\) — Markaicode](#) - [The New LangChain Architecture: langchain-core and langchain-community —](#)

[LangChain Blog - Chat Models — LangChain Documentation - PromptTemplate vs ChatPromptTemplate: Understanding and Invoking Them in LangChain — Medium](#) - [Production Pitfalls of LangChain Nobody Warns You About — CodeToDeploy](#) - [What Callbacks Are in LangChain — The Neural Base](#) - [LangChain LCEL in Practice: From Legacy Chains to Streaming Responses — BetterLink Blog](#) - [LangChain in Production: Patterns and Anti-Patterns — sph.sh](#)